Assignements

I. Business Understanding

The owner of the Bannockburn Books bookstore channel wants to create a new store while reducing the risk of creating stores with poor performance.

It is therefore a question of using the data on existing stores to be able to carry out a classsification model making it possible to predict the performance of a store.

To do this, it is available to us the "data.csv" database comprising 274 observations (stores) and 19 variables (13 quantitative and 07 qualitative). It does not have a missing value.

Terms Used

- **Model :** A statistical model is a mathematical model that embodies a set of statistical assumptions concerning the generation of sample data (and similar data from a larger population). A statistical model represents, often in considerably idealized form, the data-generating process.
  A statistical model is usually specified as a mathematical relationship between one or more random variables and other non-random variables.
- **Variable:** A statistical variable is a characteristic that can take many of the values of a set of possible observations to which a measure or quality can be applied.

- **Chi-square test:** A chi-square statistic is a test that measures how a model compares to actual observed data.
- **P-value :** the probability that a particular statistical measure, such as the mean or standard deviation, of an assumed probability distribution will be greater than or equal to (or less than or equal to in some instances) observed results

II. Data Understanding

- **Town:** town where store is located.

It's a categorical variable.

The chi-square test performed on this variable gives a p-value=38.74>0.1. This variable has no impact on the performance variable. So we do not take it into account in the rest of the analysis.

- **Country:** country where store is located.

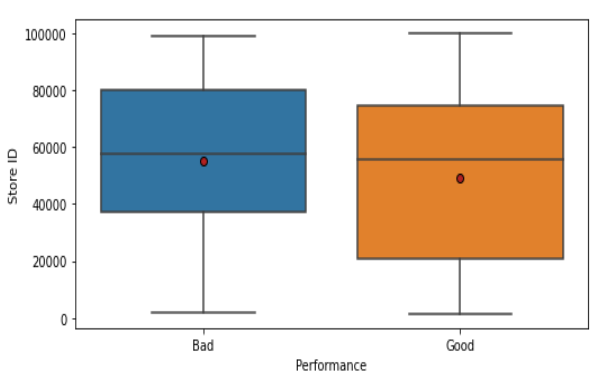
It is Categorical variable.

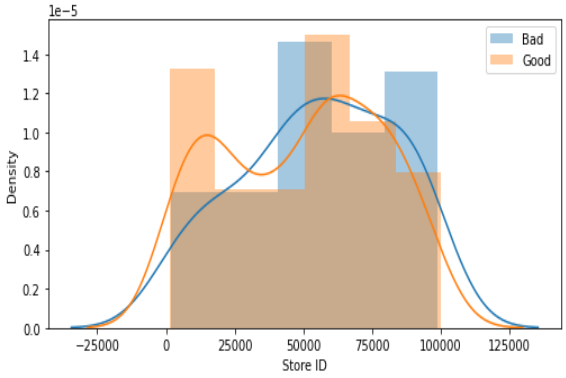
The Chi-square test gives a p-value=9.5>0.1

The variable is not taken into account in the rest of the analysis because it does not impact the performance.

- **Store ID:** unique identifier for this store.

It's numerical variable.





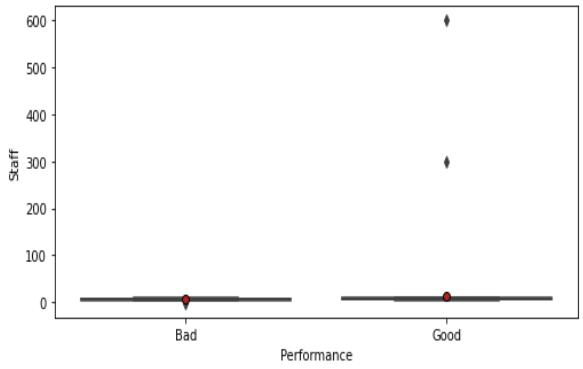
We can notice that its distribution according to the performance variable is not so different. Moreover, being a unique identifier for each store it does not influence the store performance. We will therefore not take this variable into account in the following.

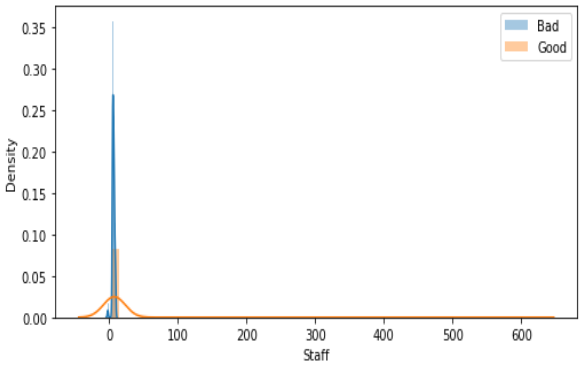
- **Manager name:** name of the lead member of staff in the store.

It's categorical variable.

We decided to do without it because it has about 176 possible values. It would therefore be complicated to apply an encoding to it.

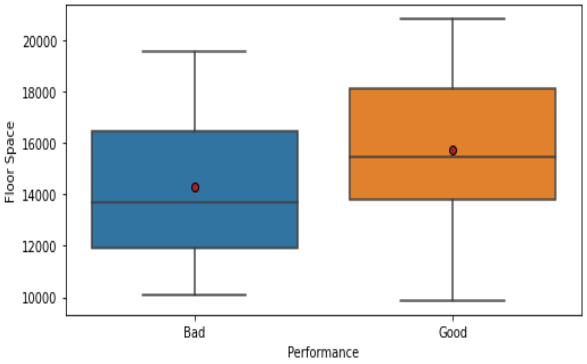
- **Staff:** number of staff employed by the store.

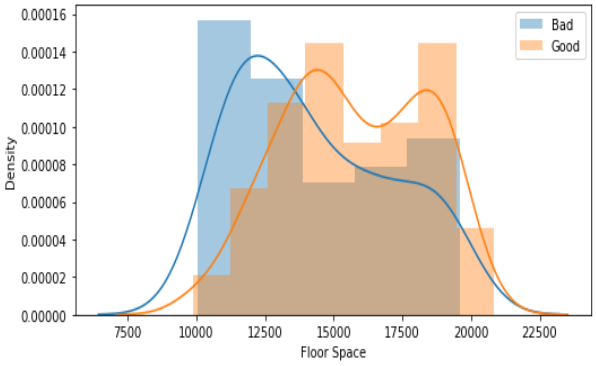




In addition to this not very explicit distribution, the correlation ratio with the performance variable is 0.34%. It's a numerical variable.

- **Floor Space:** size of store in square feet.

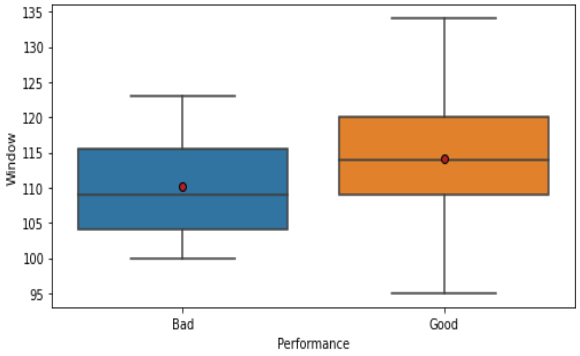


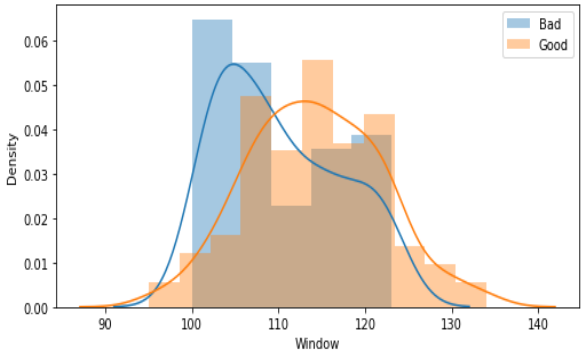


It's a numerical variable.

The correlation ratio with respect to the performance variable is 5.45%.

- **Window:** size (sq ft) of store windows suitable for displaying new books, posters, promotions etc.





It's numerical variable.

The correlation ratio with respect to the performance variable is 5.05%

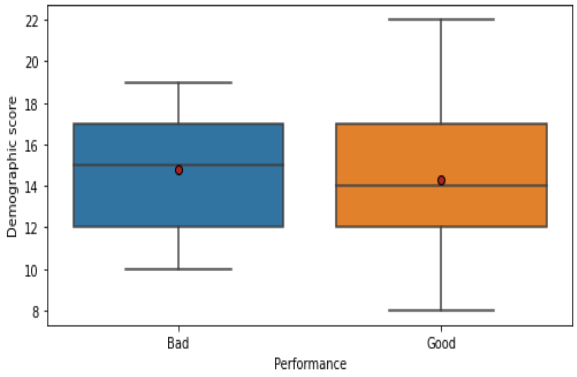
- **Car park:** does the store have a car park?.

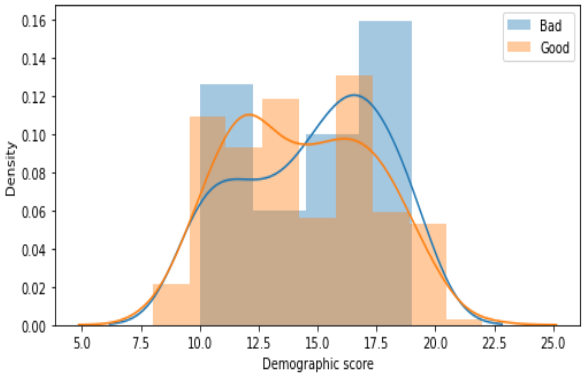
It's Categorical variable.

The Chi-square test gives a p-value=0.08 < 0.1

This variable is correlated with performance at the 10% threshold. So we decide to keep it for the following.

- **Demographic score:** a number indicating relative wealth of the people living in the area around the store.





It's numerical variable.

The correlation ratio with respect to the performance variable is 0.52%

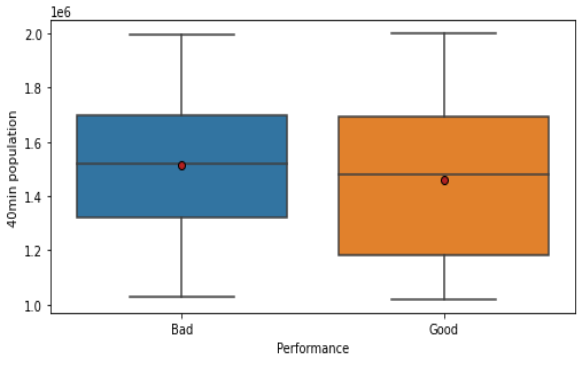
- **<u>Location:</u>** location of the store.

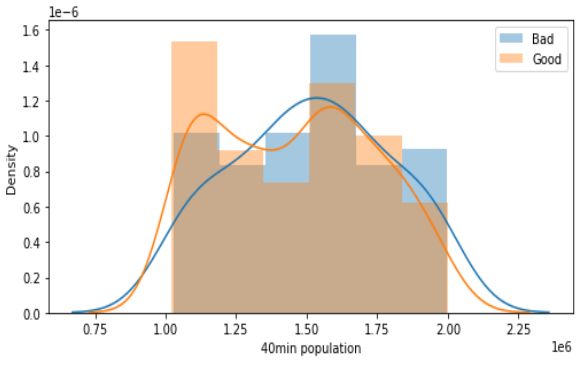
It's Categorical variable.

The Chi-square test gives a p-value=0.0006 < 0.05

This variable is correlated with performance at the 5% threshold. So we decide to keep it for the following.

- **<u>40min population:</u>** approximate number of people living within a 40 minute drive of the store.

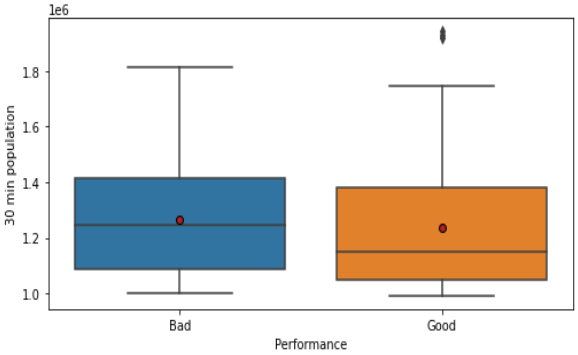


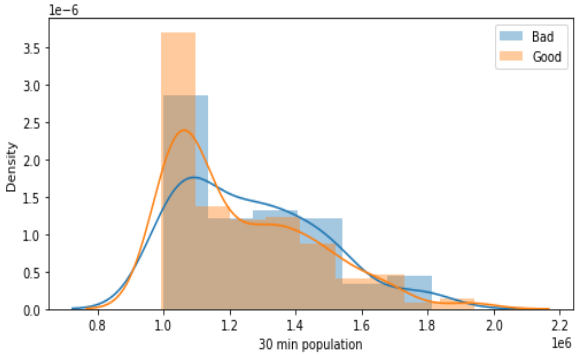


It's a numerical variable.

The correlation ratio with respect to the performance variable is 0.73%

- **<u>30 min population:</u>** approximate number of people living within a 30 minute drive of the store.

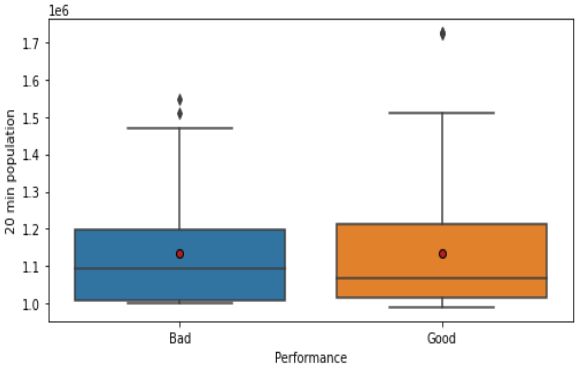


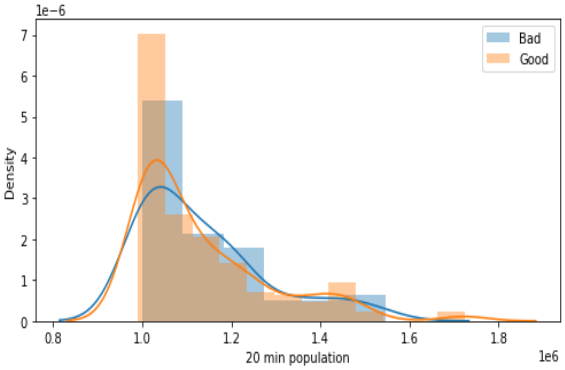


It's a numerical variable.

The correlation ratio with respect to the performance variable is 0.37%

- **<u>20 min population:</u>** approximate number of people living within a 20 minute drive of the store

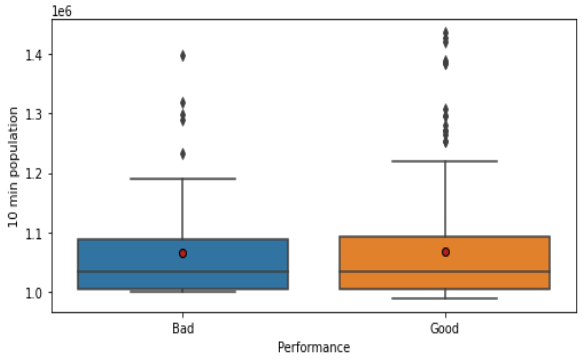


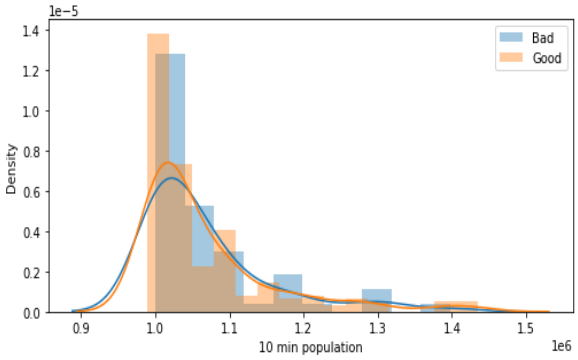


It's a numerical variable.

The correlation ratio with respect to the performance variable is 0.003%

- **<u>10 min population:</u>** approximate number of people living within a 10 minute drive of the store

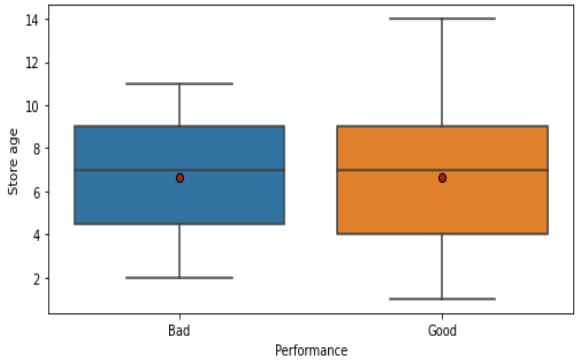


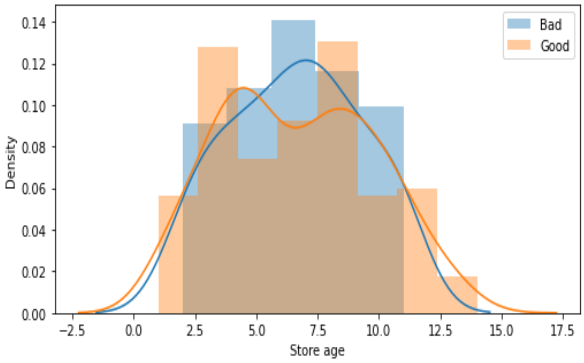


It's a numerical variable.

The correlation ratio with respect to the performance variable is 0.0048%

- **<u>Store age</u>:** in years

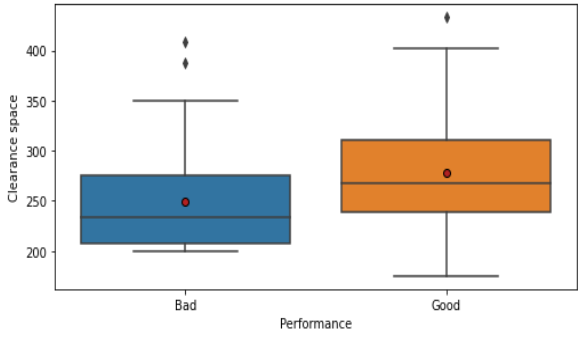


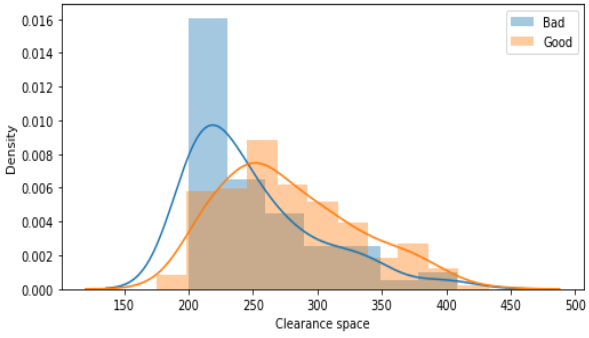


It's a numerical variable.

The correlation ratio with respect to the performance variable is 0.0049%

- **<u>Clearance space:</u>** sq feet of shelving reserved for book clearance promotions.

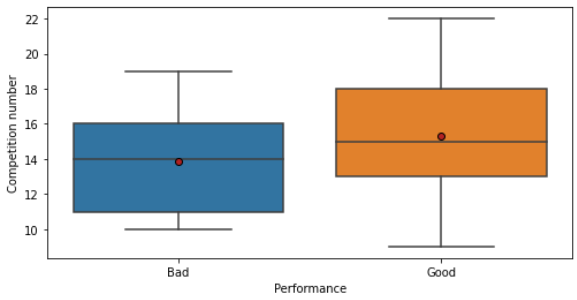


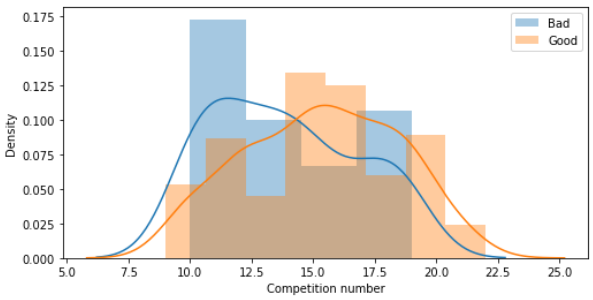


It's a numerical variable.

The correlation ratio with respect to the performance variable is 5.26%

- **Competition number:** number of competing stores within a 30 minute drive of the store.

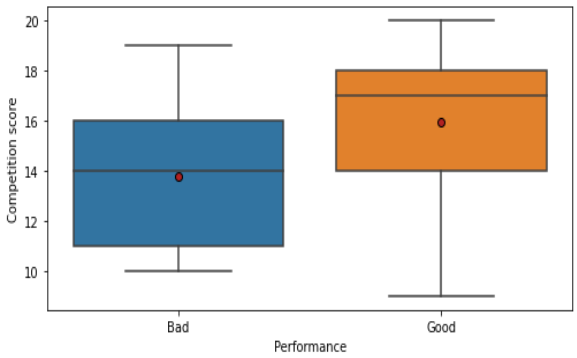


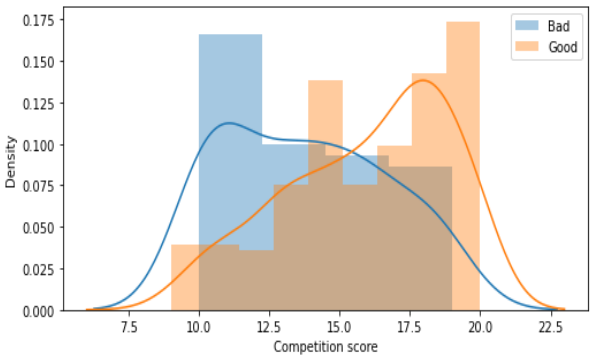


It's a numerical variable.

The correlation ratio with respect to the performance variable is 4.11%

- **Competition score:** assigned by the store's management, this is a score in the range 1-20 showing how strong on average the competing stores are.





It's a numerical variable.

The correlation ratio with respect to the performance variable is 9.63%

Rules:

We decide to keep the variables whose:

- The chi-square test gives a p-value<0.1 for the qualitative variables

- The correlation ratio is > 5%.

Hence the remaining variables are:

'Floor Space', 'Window', 'Car park', 'Location', 'Clearance space', 'Competition score', 'Performance'

III. Data Preparation

First, we make a correction on the level of the observations of the variable "Car Park" where some errors slipped in during the data collection. Indeed, we notice that instead of "Yes" or "No" it is sometimes used "Y" or "N". So we standardize the column with "Yes" and "No".

Then we encode the variable to be predicted by just using the number of labels. Which gives: Bad=0, Good=1.

After that we encode the categorical variables using the One Hot Encoding principle and we standardize the numerical variables. Standardization allows us to bring all variables to the same scale while maintaining proportionality. Here, One Hot Encoding is preferred to Label Encoding because the latter tends to give more importance to the instance that has been encoded by the most people. We therefore choose to use One Hot Encoding to compensate for this drawback.

Finally, we randomly divide the data into three samples: trainset, validationset and testset. 60% for trainset, 20% for validationset and testset.

IV. Modeling

Decision Tree

Parameters and definitions:

Criterion *{"gini", "entropy", "log_loss"}, default="gini"*

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain.

Splitter *{"best", "random"}, default="best"*

The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split.

max_depth, *default=None*

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split *default=2*

The minimum number of samples required to split an internal node:

- If int, then consider min_samples_split as the minimum number.
- If float, then min_samples_split is a fraction and ceil(min_samples_split * n_samples) are the minimum number of samples for each split.

min_samples_leaf, *default=1*

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least min_samples_leaf training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

min_weight_fraction_leaf, *default=0.0*

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.

max_features, *default=None*

The number of features to consider when looking for the best split:

random_state, *default=None*

Controls the randomness of the estimator. The features are always randomly permuted at each split, even if splitter is set to "best". When max_features < n_features, the algorithm will select max_features at random at each split before finding the best split among them. But the best found split may vary across different runs, even if max_features=n_features. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, random_state has to be fixed to an integer.

max_leaf_nodes, *default=None*

Grow a tree with max_leaf_nodes in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.

class_weight default=None

Weights associated with classes in the form {class_label: weight}. If None, all classes are supposed to have weight one.

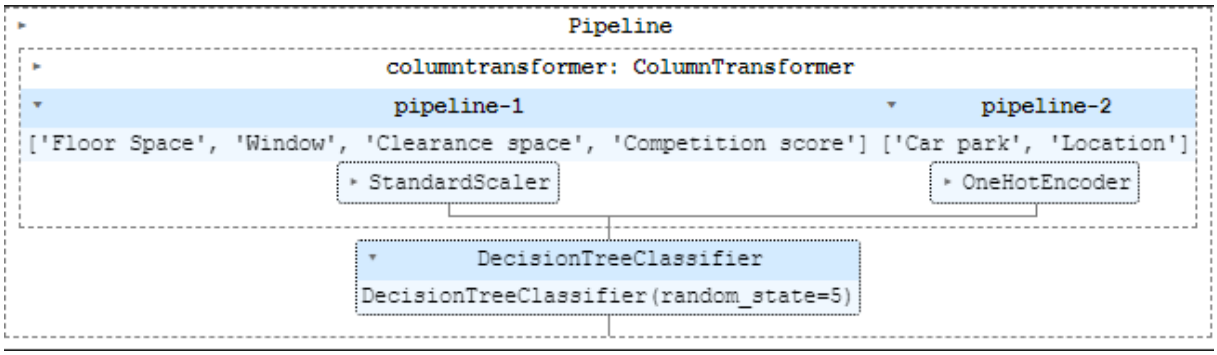
ccp_alpha, ***default=0.0***

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ccp_alpha will be chosen. By default, no pruning is performed.

Pipeline

We use a pipeline coupled with a method to find the best parameters for a model (GridSearch-CV). So we tried to find the best value for the "criterion" and "max-depth" parameters.

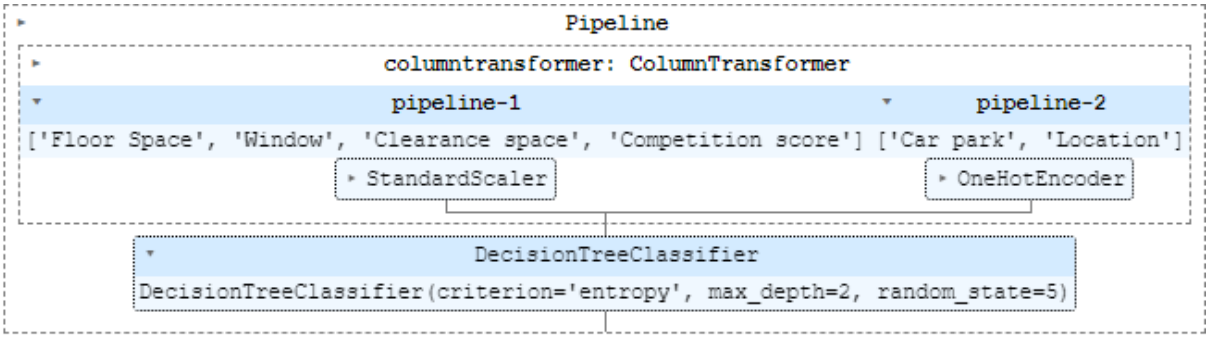
Here is the architecture of the Pipeline used:



And GridSearch-CV architecture is :

```
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('columntransformer',
                                        ColumnTransformer(transformers=[('pipeline-1',
                                                                         Pipeline(steps=[('standardscaler',
                                                                                         StandardScaler())]),
                                                                         ['Floor '
                                                                          'Space',
                                                                          'Window',
                                                                          'Clearance '
                                                                          'space',
                                                                          'Competition '
                                                                          'score']),
                                                                        ('pipeline-2',
                                                                         Pipeline(steps=[('onehotencoder',
                                                                                         OneHotEncoder())]),
                                                                         ['Car '
                                                                          'park',
                                                                          'Location'])])),
                                       ('decisiontreeclassifier',
                                        DecisionTreeClassifier(random_state=5))]),
             param_grid={'decisiontreeclassifier__criterion': ['gini',
                                                               'entropy'],
                         'decisiontreeclassifier__max_depth': array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])})
```

By combining these two architectures we obtain the one below which represents the best.



This model allowed us to have a score of 78% on the learning data base.

Logistic Regression

Parameters and definitions:

penalty*{'l1', 'l2', 'elasticnet', 'none'}, default='l2'*

Specify the norm of the penalty:

- 'none' : no penalty is added;

- “l2”: add a L2 penalty term and it is the default choice;
- “l1”: add a L1 penalty term;
- “elasticnet”: both L1 and L2 penalty terms are added.

dual, *default=False*

Dual or primal formulation. Dual formulation is only implemented for l2 penalty with liblinear solver. Prefer dual=False when n_samples > n_features.

tol, *default=1e-4*

Tolerance for stopping criteria.

C, *default=1.0*

Inverse of regularization strength; must be a positive float. Like in support vector machines, smaller values specify stronger regularization.

fit_intercept, *default=True*

Specifies if a constant (a.k.a. bias or intercept) should be added to the decision function.

intercept_scaling, *default=1*

Useful only when the solver ‘liblinear’ is used and self.fit_intercept is set to True. In this case, x becomes [x, self.intercept_scaling], i.e. a “synthetic” feature with constant value equal to intercept_scaling is appended to the instance vector. The intercept becomes intercept_scaling * synthetic_feature_weight.

Note! the synthetic feature weight is subject to l1/l2 regularization as all other features. To lessen the effect of regularization on synthetic feature weight (and therefore on the intercept) intercept_scaling has to be increased.

class_weight, *default=None*

Weights associated with classes in the form {class_label: weight}. If not given, all classes are supposed to have weight one.

The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as n_samples / (n_classes * np.bincount(y)).

Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.

random_state*int, RandomState instance, default=None*

Used when solver == 'sag', 'saga' or 'liblinear' to shuffle the data.

Solver *{'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'}, default='lbfgs'*

Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects:

- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- For multiclass problems, only 'newton-cg', 'sag', 'saga' and 'lbfgs' handle multinomial loss;
- 'liblinear' is limited to one-versus-rest schemes.

max_iter*int, default=100*

Maximum number of iterations taken for the solvers to converge.

multi_class *{'auto', 'ovr', 'multinomial'}, default='auto'*

If the option chosen is 'ovr', then a binary problem is fit for each label. For 'multinomial' the loss minimised is the multinomial loss fit across the entire probability distribution, *even when the data is binary.* 'multinomial' is unavailable when solver='liblinear'. 'auto' selects 'ovr' if

the data is binary, or if solver='liblinear', and otherwise selects 'multinomial'.

verbose, *default=0*

For the liblinear and lbfgs solvers set verbose to any positive number for verbosity.

warm_start, *default=False*

When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution. Useless for liblinear solver.

n_jobs, *default=None*

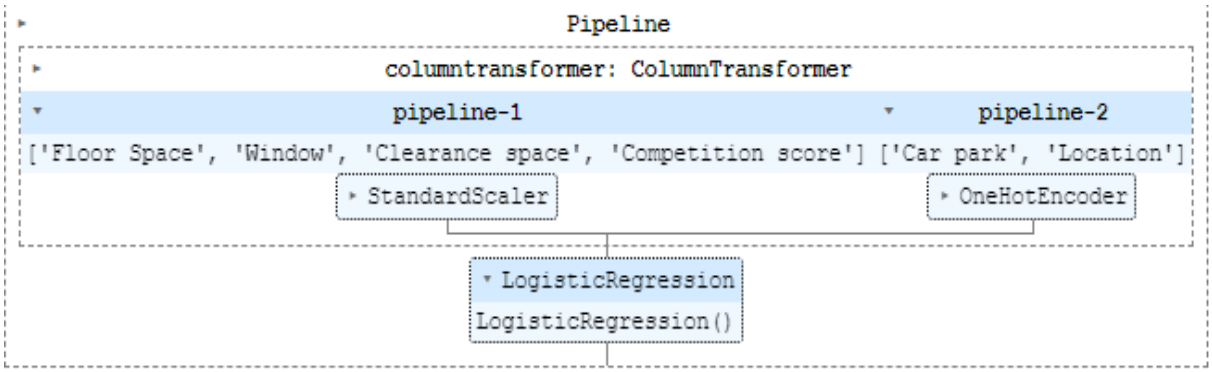
Number of CPU cores used when parallelizing over classes if multi_class='ovr'". This parameter is ignored when the solver is set to 'liblinear' regardless of whether 'multi_class' is specified or not.

l1_ratio, *default=None*

The Elastic-Net mixing parameter, with 0 <= l1_ratio <= 1. Only used if penalty='elasticnet'. Setting l1_ratio=0 is equivalent to using penalty='l2', while setting l1_ratio=1 is equivalent to using penalty='l1'. For 0 < l1_ratio <1, the penalty is a combination of L1 and L2.

Pipeline

We implement the following logistic regression pipeline and keeping the default parameters of a logistic regression model:



This model allowed us to have a score of 79% on the learning data base.

Neural Network Model

We implements a multi-layer perceptron (MLP) algorithm for model.

Parameters and definitions

hidden_layer_sizes, *length = n_layers - 2, default=(100,)*

The ith element represents the number of neurons in the ith hidden layer.

Activation *{'identity', 'logistic', 'tanh', 'relu'}, default='relu'*

Activation function for the hidden layer.

- 'identity', no-op activation, useful to implement linear bottleneck, returns f(x) = x
- 'logistic', the logistic sigmoid function, returns f(x) = 1 / (1 + exp(-x)).
- 'tanh', the hyperbolic tan function, returns f(x) = tanh(x).
- 'relu', the rectified linear unit function, returns f(x) = max(0, x)

Solver *{'lbfgs', 'sgd', 'adam'}, default='adam'*

The solver for weight optimization.

- 'lbfgs' is an optimizer in the family of quasi-Newton methods.

- ‘sgd’ refers to stochastic gradient descent.
- ‘adam’ refers to a stochastic gradient-based optimizer proposed by Kingma, Diederik, and Jimmy Ba

Note: The default solver ‘adam’ works pretty well on relatively large datasets (with thousands of training samples or more) in terms of both training time and validation score. For small datasets, however, ‘lbfgs’ can converge faster and perform better.

alpha, *default=0.0001*

Strength of the L2 regularization term. The L2 regularization term is divided by the sample size when added to the loss.

batch_size, *default=’auto’*

Size of minibatches for stochastic optimizers. If the solver is ‘lbfgs’, the classifier will not use minibatch. When set to “auto”, batch_size=min(200, n_samples).

learning_rate *{‘constant’, ‘invscaling’, ‘adaptive’}, default=’constant’*

Learning rate schedule for weight updates.

- ‘constant’ is a constant learning rate given by ‘learning_rate_init’.
- ‘invscaling’ gradually decreases the learning rate at each time step ‘t’ using an inverse scaling exponent of ‘power_t’. effective_learning_rate = learning_rate_init / pow(t, power_t)
- ‘adaptive’ keeps the learning rate constant to ‘learning_rate_init’ as long as training loss keeps decreasing. Each time two consecutive epochs fail to decrease training loss by at least tol, or fail to increase validation score by at least tol if ‘early_stopping’ is on, the current learning rate is divided by 5.

Only used when solver='sgd'.

learning_rate_init, *default=0.001*

The initial learning rate used. It controls the step-size in updating the weights. Only used when solver='sgd' or 'adam'.

power_t, *default=0.5*

The exponent for inverse scaling learning rate. It is used in updating effective learning rate when the learning_rate is set to 'invscaling'. Only used when solver='sgd'.

max_iter, *default=200*

Maximum number of iterations. The solver iterates until convergence (determined by 'tol') or this number of iterations. For stochastic solvers ('sgd', 'adam'), note that this determines the number of epochs (how many times each data point will be used), not the number of gradient steps.

shuffle, *default=True*

Whether to shuffle samples in each iteration. Only used when solver='sgd' or 'adam'.

random_state, *RandomState instance, default=None*

Determines random number generation for weights and bias initialization, train-test split if early stopping is used, and batch sampling when solver='sgd' or 'adam'. Pass an int for reproducible results across multiple function calls.

tol, *default=1e-4*

Tolerance for the optimization. When the loss or score is not improving by at least tol for n_iter_no_change consecutive iterations, unless learning_rate is set to 'adaptive', convergence is considered to be reached and training stops.

Verbose , *default=False*

Whether to print progress messages to stdout.

warm_start, *default=False*

When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution.

momentum, *default=0.9*

Momentum for gradient descent update. Should be between 0 and 1. Only used when solver='sgd'.

nesterovs_momentum, *default=True*

Whether to use Nesterov's momentum. Only used when solver='sgd' and momentum > 0.

early_stopping, *default=False*

Whether to use early stopping to terminate training when validation score is not improving. If set to true, it will automatically set aside 10% of training data as validation and terminate training when validation score is not improving by at least tol for n_iter_no_change consecutive epochs. The split is stratified, except in a multilabel setting. If early stopping is False, then the training stops when the training loss does not improve by more than tol for n_iter_no_change consecutive passes over the training set. Only effective when solver='sgd' or 'adam'.

validation_fraction, *default=0.1*

The proportion of training data to set aside as validation set for early stopping. Must be between 0 and 1. Only used if early_stopping is True.

beta_1, *default=0.9*

Exponential decay rate for estimates of first moment vector in adam, should be in [0, 1). Only used when solver='adam'.

beta_2, *default=0.999*

Exponential decay rate for estimates of second moment vector in adam, should be in [0, 1). Only used when solver='adam'.

epsilon, *default=1e-8*

Value for numerical stability in adam. Only used when solver='adam'.

n_iter_no_change, *default=10*

Maximum number of epochs to not meet tol improvement. Only effective when solver='sgd' or 'adam'.

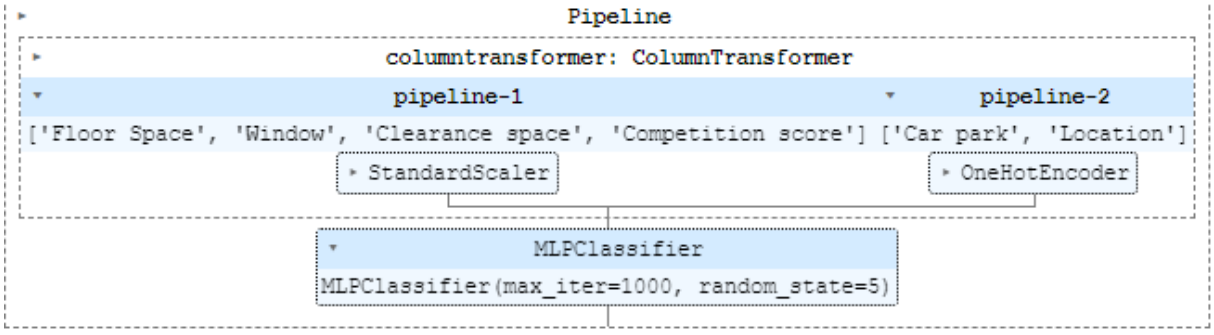
max_fun, *default=15000*

Only used when solver='lbfgs'. Maximum number of loss function calls. The solver iterates until convergence (determined by 'tol'), number of iterations reaches max_iter, or this number of loss function calls. Note that number of loss function calls will be greater than or equal to the number of iterations for the MLPClassifier.

Pipeline

We just use the default settings.

The pipeline used is:



This model allowed us to have a score of 89% on the learning data base.

In conclusion, chose the neural network model because it has a better performance.

V. Evaluation

We obtain the following confusion matrix for the test data.

The score achieved is 85%.

